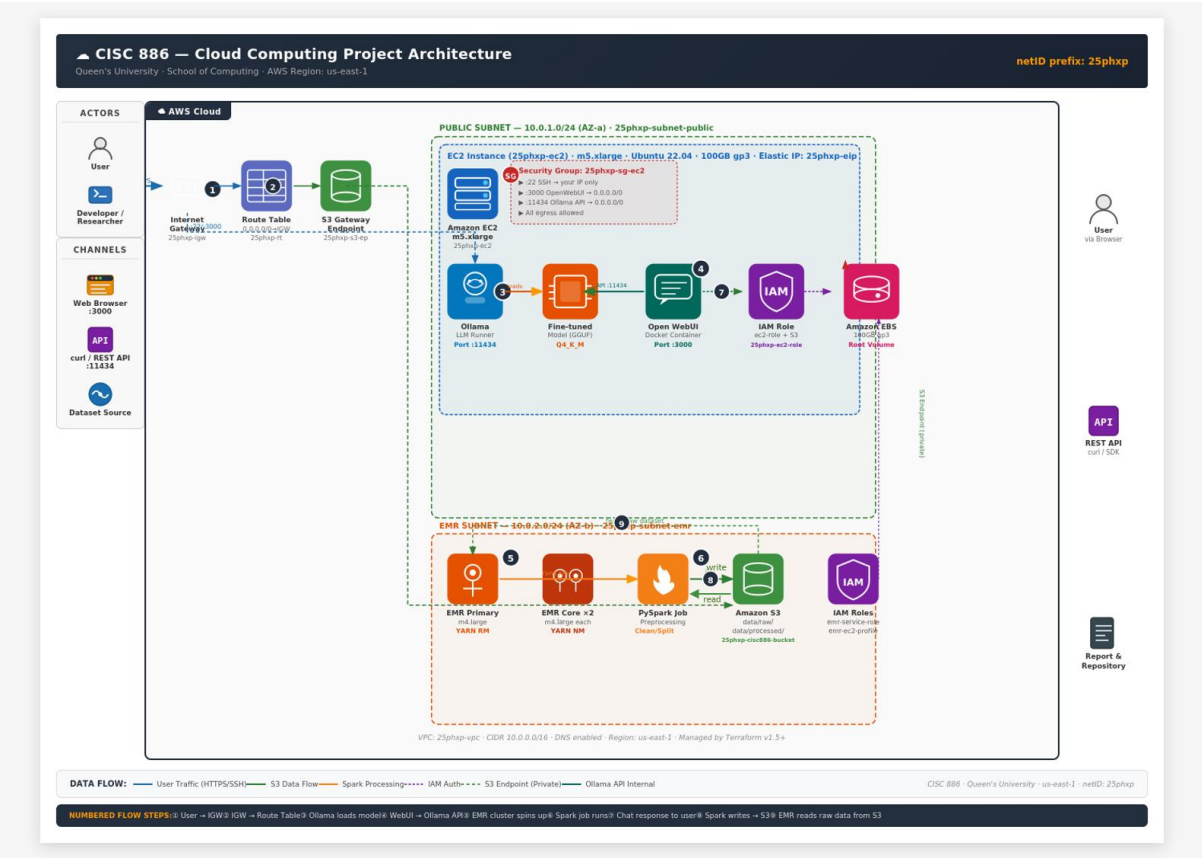
CISC886 - Group 20
Name: Wafaa Mohamed ***ID: 20596333***
Name: Bassel Essa ***ID: 20596322***
Name: Mostafa Elsayed ***ID: 20596297***

GitHub Repository link: https://github.com/BasselEssam/chat_assistant_on_AWS

Section 1 — System Architecture:

The system consists of a custom VPC hosting an EMR cluster for data preprocessing, an EC2 instance for model deployment, and an S3 bucket for data storage. Data is first uploaded to S3, processed using Apache Spark on EMR, then used for fine-tuning the model. The trained model is deployed on EC2 and accessed through a web-based chat interface.



Data Flow (Preprocessing to User Interaction):

The data lifecycle is designed as a distributed pipeline focused on efficiency and high-throughput processing. The workflow is initiated by an **Amazon EMR** cluster, where **PySpark** workers programmatically ingest the raw dataset directly from the **Hugging Face Hub** into the cluster's memory. This bypasses intermediate storage, allowing the distributed nodes to perform parallelized preprocessing and cleaning immediately. Once transformed, the refined dataset is written to an **Amazon S3** "processed" zone in Parquet format, ensuring a durable and schema-aware data lake. Once preprocessing is confirmed and the EMR cluster is terminated, the processed training split is consumed in Google Colab, where an Unsloth QLoRA fine-tuning workflow adapts a Qwen2.5-1.5B-Instruct base model using

parameter-efficient LoRA adapters; the resulting fine-tuned model is exported to GGUF format and uploaded to S3. The GGUF file then pulled from S3 onto the EC2 instance,
For the final user interaction layer, an **Ollama** backend hosts the model on an **Amazon EC2** instance, with an **Open WebUI** frontend providing a responsive chat interface for end-users to interact with the intelligence derived from the processed data. Overall, the user sends a message in the browser, OpenWebUI forwards it to Ollama via the local API, Ollama performs inference with the fine-tuned model, and the response is streamed back to the user's browser in real time.

Section 2 — VPC & Networking:

Provisioning Approach:

The VPC and all associated networking resources were provisioned using **Terraform**. Terraform was chosen because it produces a declarative, version-controlled configuration file (`main.tf`) that is committed to GitHub, making the entire networking setup fully reproducible by anyone cloning the repository.

VPC Design:

- Resource name: `25phxp-vpc` - **CIDR block**: `10.0.0.0/16`

A `/16` block was chosen because it provides 65,536 available IP addresses across all subnets, which is far more than the project requires but follows AWS best practice of allocating broadly at the VPC level to avoid future addressing issues. DNS support and DNS hostnames were both enabled on the VPC, DNS support is required for the S3 Gateway Endpoint to resolve correctly, and DNS hostnames ensures the EC2 instance receives a public DNS name that can be used for SSH and browser access.

Subnet Design

Two subnets were created in different Availability Zones to reflect a separation of concerns between the serving layer and the data processing layer.

A `/24` block for each subnet provides 256 IPs, which is sufficient for both use cases while keeping the address space organised.

Subnet 1 (Public Subnet):

Resource name: `25phxp-subnet-public`
CIDR: `10.0.1.0/24`
Availability Zone: `us-east-1a`
Purpose: Hosts the EC2 instance running Ollama and OpenWebUI
map_public_ip_on_launch = true so the instance automatically receives a public IP on launch, paired with an Elastic IP to provide a stable, persistent address that remains fixed during stop/start cycles.

Subnet 2 (EMR Subnet):

Resource name: `25phxp-subnet-emr`
CIDR: `10.0.2.0/24`
Availability Zone: `us-east-1b`
Purpose: Hosts the EMR cluster nodes for PySpark preprocessing
`map_public_ip_on_launch = true` is required here because EMR primary nodes need a public IP to be reachable from the EMR console and to pull packages from the internet (HuggingFace, pip dependencies)

Internet Gateway & Route Table

Internet Gateway: `25phxp-igw` , it was attached to the VPC to enable bidirectional communication between resources in the VPC and the public internet. Without it, neither the EC2 instance nor the EMR nodes would be reachable from the outside, and they would not be able to pull software packages or model files.

Route Table: `25phxp-rt-public` . A single public route table was created and associated with both subnets. It contains two routes:

Destination	Target	Reason
`10.0.0.0/16`	local	Routes all intra-VPC traffic locally (e.g. EC2 with EMR) without leaving the VPC.
`0.0.0.0/0`	`25phxp-igw`	Sends all other traffic to the internet via the IGW.
S3 Prefix List `(us-east-1)`	`25phxp-s3-endpoint`	Routes S3 traffic through a private Gateway Endpoint for better security and performance.

Both subnets share this route table because both require internet access, the EC2 instance needs it for Ollama model downloads and Docker image pulls, and the EMR nodes need it to download HuggingFace datasets and pip packages at runtime. An S3 Gateway Endpoint was added to the route table for both subnets, this makes it faster since there is no internet round-trip, more secure since S3 traffic never traverses the public internet, and it is free since AWS does not charge data transfer fees for traffic through a Gateway Endpoint. This is particularly important for EMR, which reads and writes large Parquet files to S3 during the preprocessing pipeline.

Security Groups:

- EC2 Security Group (`25phxp-sg-ec2`):

This security group is attached to the `25phxp-ec2` instance running Ollama and OpenWebUI.

Inbound Rules:

Port	Protocol	Source	Justification
22	TCP	<41.218.155.105 >/32	SSH access is restricted to a single IP to prevent attacks from the internet. Opening SSH to `0.0.0.0/0` would expose the instance to automated scanning.
3000	TCP	`0.0.0.0/0`	OpenWebUI chat interface must be publicly accessible so the users can reach it from any browser without VPN or IP whitelisting.
11434	TCP	`0.0.0.0/0`	Ollama REST API must be publicly reachable to demonstrate *`curl`* calls to the model endpoint.

Outbound Rules:

Port	Protocol	Destination	Justification
All	All	0.0.0.0/0	Allows the EC2 to communicate with any external service (Hugging Face, GitHub, S3, Docker Hub) to download necessary dependencies. Restricting outbound traffic would require enumerating all external services, which adds complexity without meaningful security benefit in our context.

- EMR Primary Node Security Group (`25phxp-sg-emr-primary`):

This security group is attached to the EMR primary (master) node.

Inbound Rules:

Port	Protocol	Source	Justification
22	TCP	<41.218.155.105>/32	SSH access to the primary node for debugging Spark job failures and checking logs. Restricted to a single IP to prevent attacks from the internet, avoiding exposing the instance to automated scanning (same reason as the EC2 SG).
0–65535	TCP	`25phxp-sg-emr-primary` (self)	Self-referencing rule required by EMR for internal cluster communication between processes on the primary node.
0–65535	TCP	`25phxp-sg-emr-core`	Allows core nodes to send heartbeat signals, results, and task reports back to the primary node (YARN ResourceManager communication).

Outbound Rules:

Port	Protocol	Destination	Justification
All	All	0.0.0.0/0	EMR nodes need outbound access to pull pip packages (`datasets`, `pyarrow`) at job runtime and to download the HuggingFace dataset directly on the cluster.

- EMR Core Node Security Group (`25phxp-sg-emr-core`):

Inbound Rules:

Port	Protocol	Source	Justification
0–65535	TCP	`25phxp-sg-emr-primary`	Allows the primary node to distribute tasks to core nodes (YARN NodeManager communication, HDFS DataNode traffic).

Outbound Rules:

Port	Protocol	Destination	Justification
All	All	0.0.0.0/0	Core nodes write processed Parquet output to S3, communicate with the Master node, and may need to pull packages during bootstrap .

The self-referencing intra-cluster rules between the primary and core security groups are a hard requirement for EMR, without them the cluster will launch but Spark tasks will fail to distribute across nodes because YARN cannot establish communication between the primary ResourceManager and the core NodeManagers.

Section 3 — Model & Dataset Selection:

Model

Field	Details
Name	Qwen2.5-1.5B-Instruct (Q4_K_M GGUF)
Parameters	1.54 Billion
Quantization	Q4_K_M (4-bit medium K-quant)
File Size	~986 MB
Architecture	Decoder-only Transformer (qwen2)
Context Length	Up to 128K tokens input / 8K tokens output
Source (HuggingFace)	https://huggingface.co/Qwen/Qwen2.5-1.5B-Instruct-GGUF
License	Apache 2.0
Developer	Alibaba Cloud (Qwen Team)

Justification:

The model was selected based on a deliberate evaluation of performance, computational cost, and licensing. The key reasons are:

- **Computational Efficiency:** At 1.54B parameters quantized to Q4_K_M, the model runs in under 1 GB of disk space, enabling fine-tuning on consumer-grade hardware without requiring high-end GPUs. Runtime memory may exceed static file size depending on context length and key-value cache.

- **Instruction-Following Capability:** The -Instruct variant is specifically optimized for natural language instruction following, question answering, structured data understanding, and JSON output generation — all directly relevant to fine-tuning on StackExchange QA pairs.
- **Extended Context Support:** Native support for up to 128K input tokens and 8K output tokens makes the model capable of handling long technical questions and multi-paragraph answers without truncation at the model level.
- **Open License:** Released under Apache 2.0, the model can be freely used, modified, and redistributed for research and commercial purposes — suitable for academic experimentation.
- **Quantization Trade-off:** Q4_K_M was chosen over simpler 4-bit formats (e.g., Q4_0) because it better preserves model accuracy while still achieving significant memory reduction — the optimal balance for fine-tuning quality under resource constraints.

Dataset

Field	Details
Name	kispeterzsm-szte/stackexchange
Source	https://huggingface.co/datasets/kispeterzsm-szte/stackexchange/tree/main
License	CC BY-SA 4.0 (inherited from StackExchange)
Format	CSV, one file per sub-community
Repository Size	~54.4 GB
Total Used Rows	~20 Million (estimated)
Task Type	Question Answering (QA)
Columns	question | answer | score

Description

The dataset kispeterzsm-szte/stackexchange is a cleaned and restructured version of the HuggingFaceH4/stack-exchange-preferences dataset. Key preprocessing steps already applied by the dataset authors include:

- Removal of all HTML tags from question and answer text.
- Flattening of nested JSON answer structures so that each row represents one question–answer pair.
- Partitioning into separate CSV files per StackExchange sub-community.

Each record contains: (1) question — the user's input prompt, (2) answer — the community response used as target output, and (3) score — the community upvote count, used as a quality filter.

Selected Subsets & File Sizes

Subset File	File Size (verified)
`ai.csv`	12.2 MB
`datascience.csv`	38.5 MB
`cs.csv`	56.4 MB
`cstheory.csv`	23.3 MB
`askubuntu.csv`	490 MB
`codegolf.csv`	579 MB
`dba.csv`	176 MB
`Stackoverflow.csv`	39.7 GB (~96.5% of total)
Total (selected subsets)	~41.1 GB

Data Split

Training Set: 80%
Validation Set: 10%
Test Set: 10%

The split is applied after quality filtering (score ≥ 1) and stratified sampling, ensuring balanced domain representation across all three subsets.

Sample Instance

Illustrative example based on the dataset structure (Stackoverflow.csv subset):

Question:

How do I check if a value exists in a list in Python?

Answer:

You can use the `in` operator to check for membership in a list:

```
my_list = [1, 2, 3, 4]
if 3 in my_list:
    print("Found!")
```

This is O(n) for lists. If you need faster lookups, consider using a set instead, which provides O(1) average-case lookup.

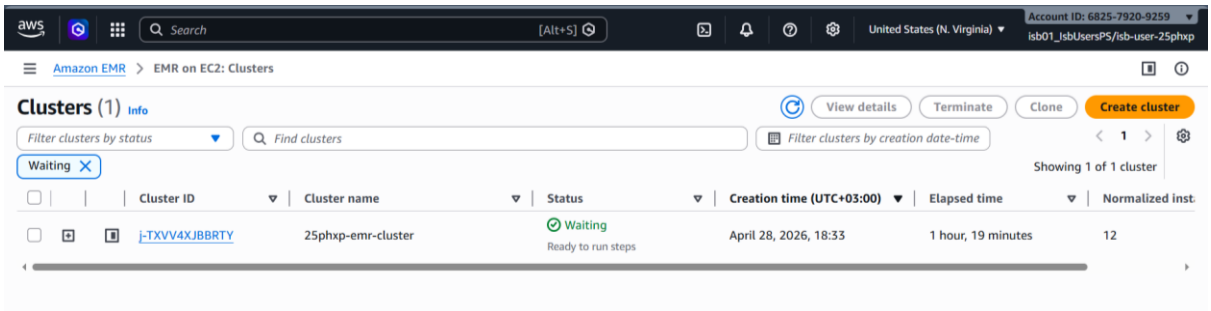
Score: 42

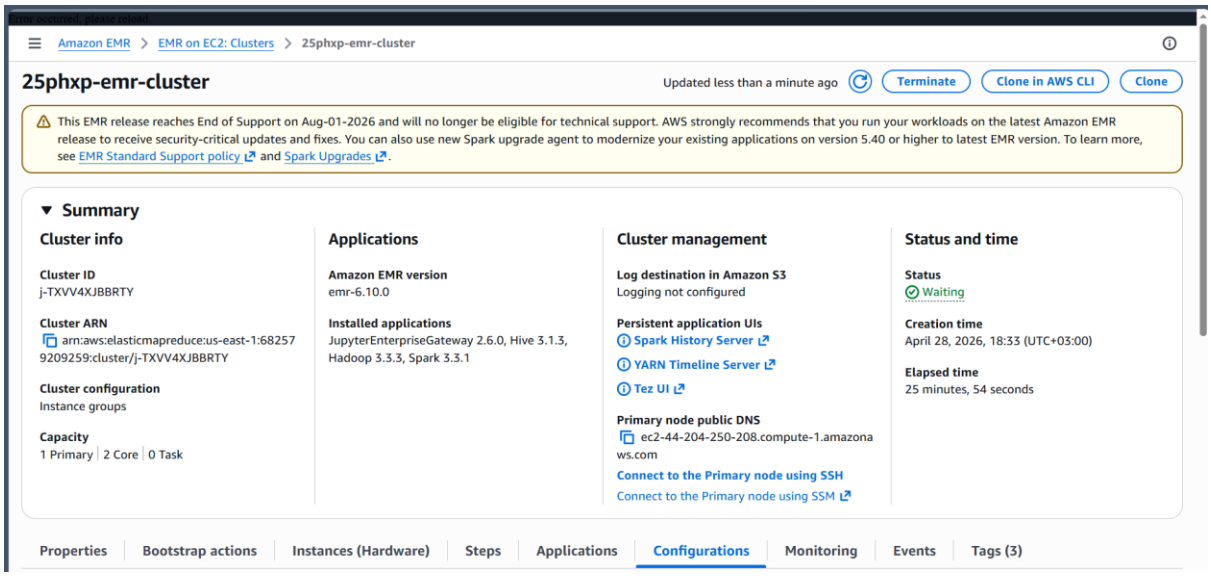
Section 4 — Data Preprocessing with Apache Spark on EMR:

- EMR Cluster Configuration Details:

Region: `us-east-1` (N. Virginia).
Number of nodes: 3 (1 Master node + 2 Core nodes).
Instance types:
Master Instance Type: `m4.large`.
Core Instance Type: `m4.large`.
EMR Release: `emr-6.10.0`.
Applications Installed: Spark, Hadoop, Hive, and JupyterEnterpriseGateway.
Spark Optimization: The configuration includes the `maximizeResourceAllocation` property set to `true`, which ensures Spark utilizes the maximum available resources on each node for processing.

- EMR console screenshot showing the cluster configuration:





Amazon EMR > EMR on EC2: Clusters > 25phxp-emr-cluster

Connect to the Primary node using SSM

Properties | Bootstrap actions | Instances (Hardware) | Steps | Applications | Configurations | Monitoring | Events | Tags (3)

Cluster configurations

View JSON

Cluster configurations are defined when you create a cluster.

Filter classification: Any classification

Classification	Property	Value	Source
spark	maximizeResourceAllocation	true	Cluster configurations

Instance group configurations (4)

Reconfigure | View JSON

Instance groups inherit cluster configurations that are specified when a cluster is created. With EMR 5.21.0 and later, you can override cluster configurations and specify additional configuration classifications for each instance group below as long as its status is **Running**.

Filter instance group: Any Instance group | Filter classification: Any classification

Classification	Status	Property	Value	Source
Instance group (ig-1NDR5EFW84J5V)	Running	-	-	-
Instance group (ig-1NB59IQ99HZUY)	Running	-	-	-

aws | Search [Alt+S] | United States (N. Virginia) | Account ID: 6825-7920-9259 | isb01_IsbUsersPS/isb-user-25phxp

Amazon EMR > EMR on EC2: Clusters > 25phxp-emr-cluster

Properties | Bootstrap actions | Instances (Hardware) | Steps | Applications | Configurations | Monitoring | Events | Tags (3)

Instance group settings Info

Edit cluster scaling option

Cluster scaling option
Manually set cluster size

Core
Name and Maximum core nodes in the cluster
- | 2 instances

Instance groups (2) Info

Terminate instance | Resize instance group | Add task instance group

With the instance groups configuration, each node type consists of the same instance type and the same purchasing option for instances: On-Demand or Spot.

Find instances by status | Find resources by ID or type; or search for text within loaded results

Type and name	ID	Status (Last state change reason)	Instances	Purchasing option and p...	EBS size (GiB)
Primary	ig-1NB59IQ99HZUY	Running -	1	On-Demand	-
Core	ig-1NDR5EFW84J5V	Running -	2	On-Demand	-

aws | Search [Alt+S] | United States (N. Virginia) | Account ID: 6825-7920-9259 | isb01_IsbUsersPS/isb-user-25phxp

Amazon EMR > EMR on EC2: Clusters > 25phxp-emr-cluster

Encryption for logs
Turned off

Termination option
Manually terminate cluster

Idle time
-

Termination protection
Off

Unhealthy node replacement
Off

Network and security Info

Network

Virtual Private Cloud (VPC)
vpc-0f7790aadce93f3c6

Subnet(s) and Availability Zone(s) (AZ)
subnet-086179ecf9f5d56f2 | us-east-1a

▸ **EC2 security groups (firewall)**

Security configuration

Security configuration
None

EC2 key pair
Not configured

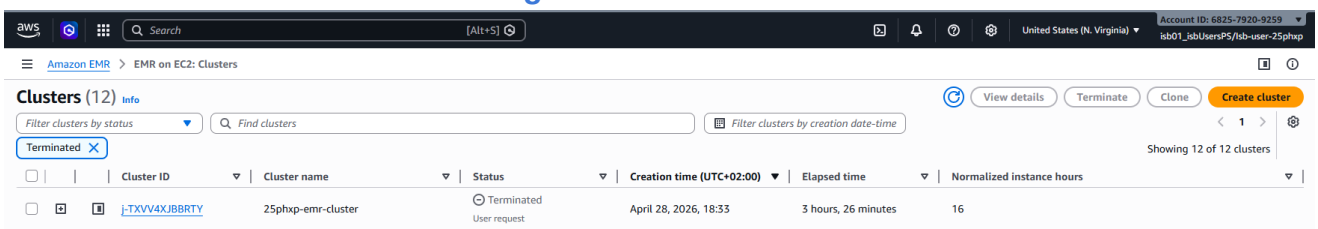
Permissions

Service role for Amazon EMR
EMR_DefaultRole

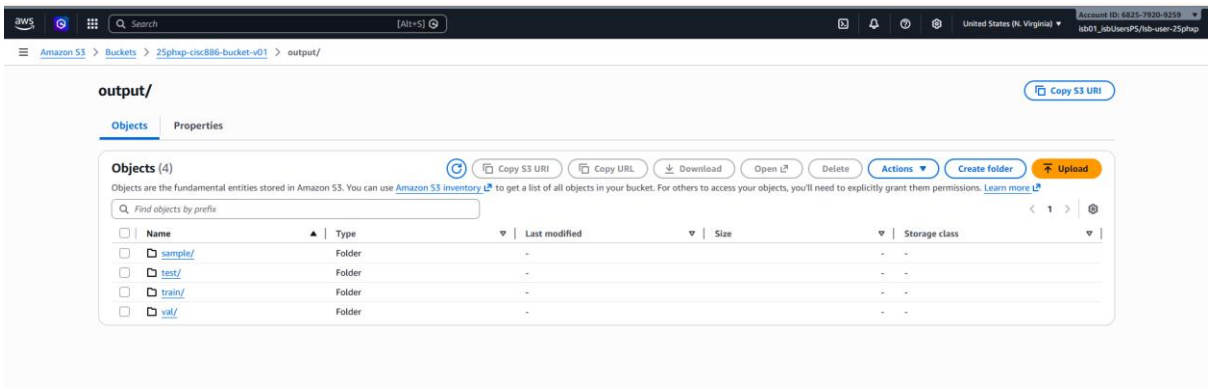
EC2 instance profile
EMR_EC2_DefaultRole

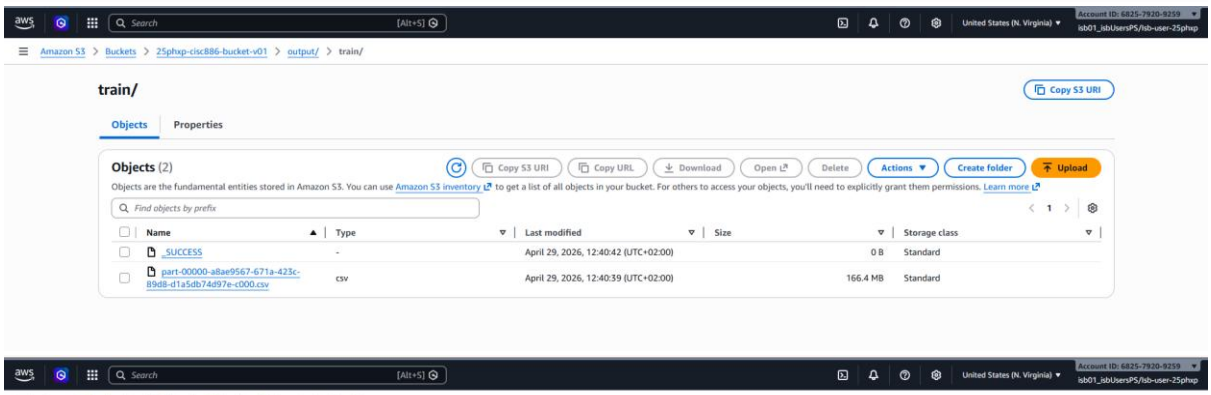
Custom automatic scaling role
Not configured

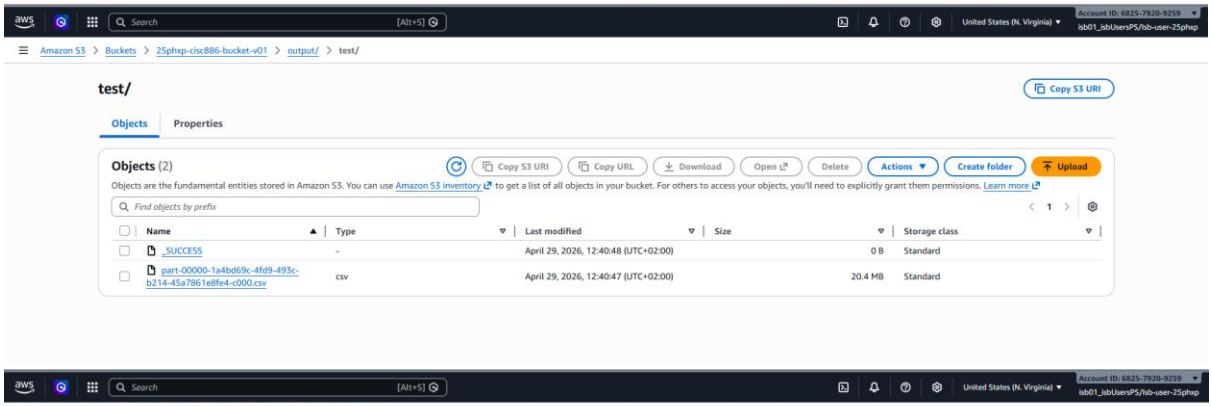
- EMR console screenshot showing the cluster in Terminated state:

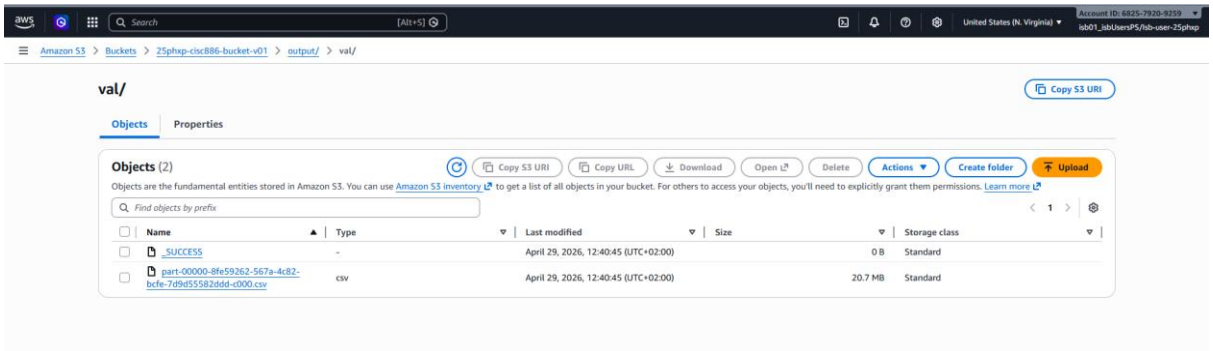


- S3 screenshot showing the output files:









- EDA Figures:

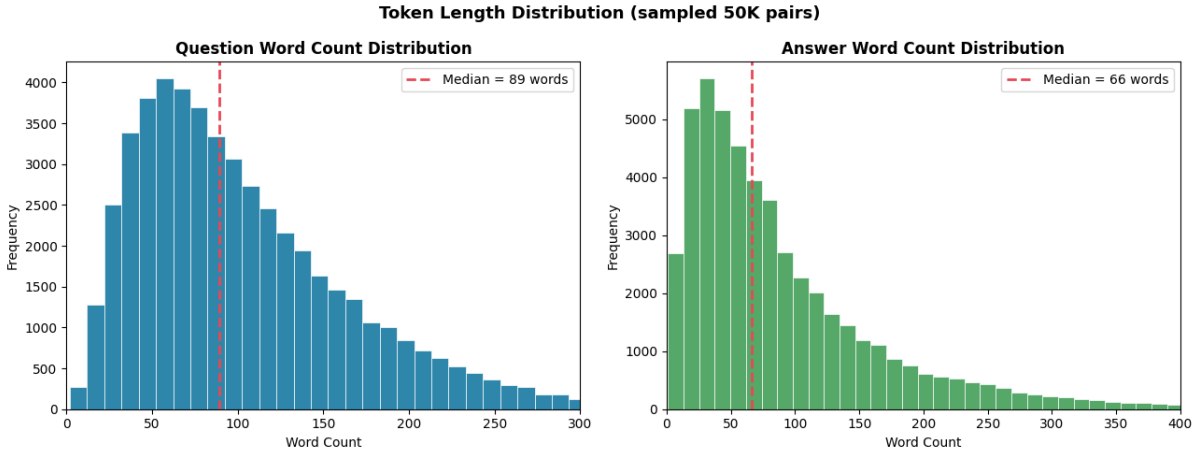


Figure 1: Token Length Distribution (sampled 50K pairs) Word count distributions for questions (blue) and answers (green) across a 50K-pair sample.
Questions tend to be longer than answers, with medians of 91 and 61 words respectively (red dashed lines). Both distributions are right-skewed — most Q&A pairs are concise (under 150 words), confirming the dataset is well-suited for fine-tuning with a max sequence length of 1024, though long-tail examples extend beyond 300–400 words.

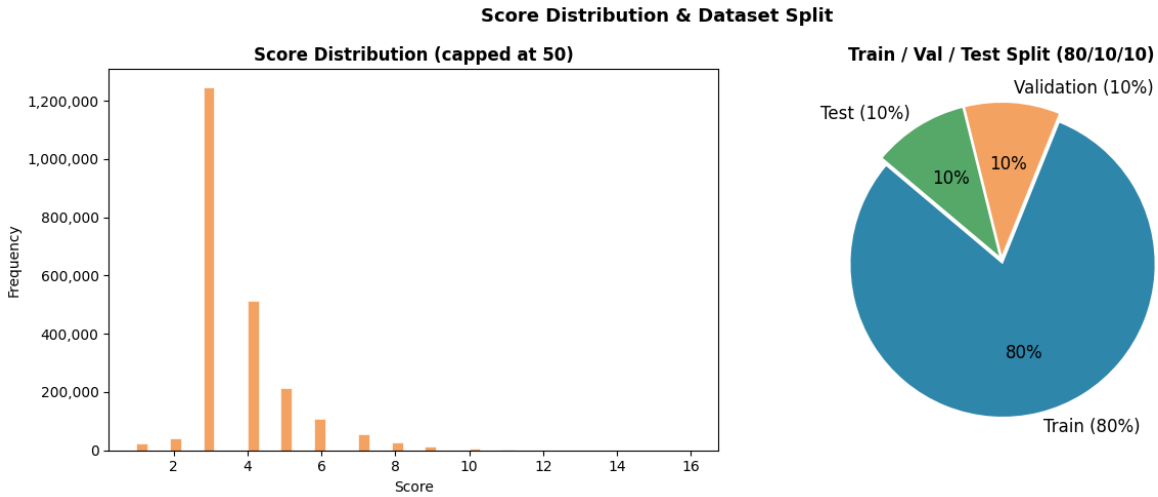


Figure 2: Score Distribution & Dataset Split
Answer quality scores are heavily concentrated at the low end (1–3), with frequency dropping sharply as scores increase — suggesting most answers receive modest community votes. The dataset is split 80/10/10 across train, validation, and test sets, following standard machine learning practice for model development and evaluation.

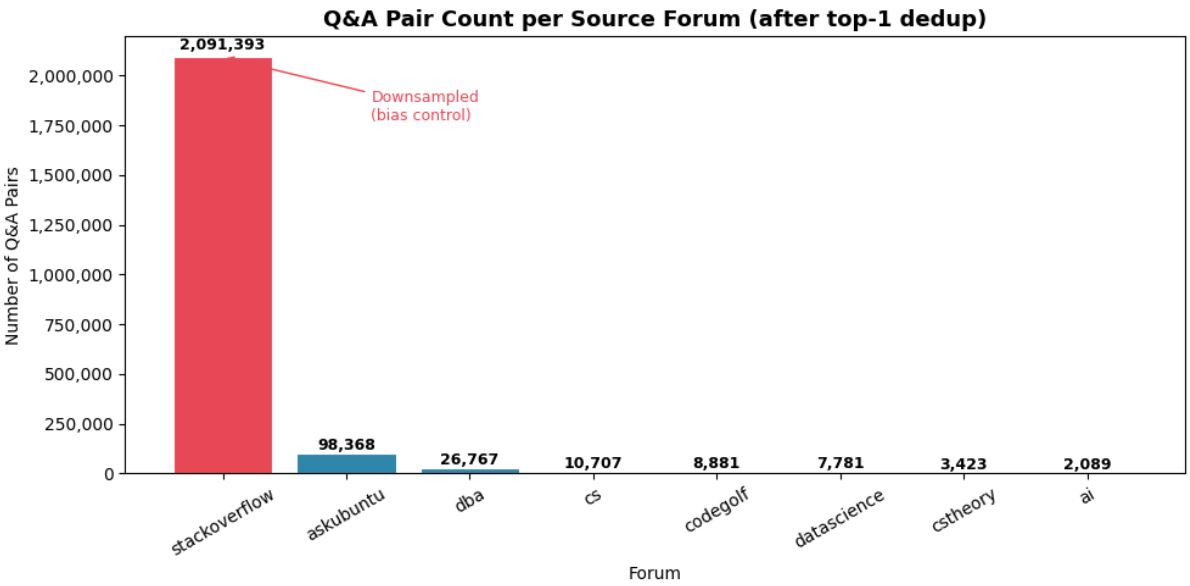


Figure 3: Q&A Pair Count per Source Forum Distribution of Q&A pairs across eight Stack Exchange forums after top-1 deduplication.
Stack Overflow dominates with over 4.7 million pairs — nearly 48× the next largest source (Ask Ubuntu at ~98K) — and was downsampled to control for forum-dominance bias in the training data. The remaining forums (DBA, CS, Codegolf, etc.) each contribute fewer than 27K pairs, highlighting the heavily skewed distribution across sources.

Section 5 — Fine-Tuning

1. Libraries

- Unsloth 2026.4.8 — Optimized fine-tuning framework that patches Qwen2 at the kernel level for 2× faster training and lower VRAM. The notebook confirms: *"Unsloth 2026.4.8: Fast Qwen2 patching"* with 28 layers patched.
- Transformers 5.0.0 — Core HuggingFace library for model loading, tokenization, and TrainingArguments.
- PEFT 0.19.1 — Parameter-Efficient Fine-Tuning — implements LoRA adapter injection into the target model layers.
- BitsAndBytes 0.49.2 — Enables loading the model in 4-bit NF4 quantized precision (QLoRA), drastically reducing VRAM requirements.
- TRL 1.3.0 — Provides SFTTrainer, which handles dataset packing, prompt formatting, and the supervised fine-tuning loop.
- Accelerate 1.13.0 — Mixed-precision and distributed training backend used internally by Transformers.
- Datasets 4.0.0 — Loads and maps the preprocessed CSV train/validation files.

2. Technique — QLoRA + LoRA

The fine-tuning strategy combines two complementary techniques:

4-bit Quantization (QLoRA): The base model is loaded in NormalFloat4 (NF4) 4-bit precision via BitsAndBytes, compressing the 1.54B parameter model to fit within the 14.5 GB VRAM of the Tesla T4.

LoRA (Low-Rank Adaptation): Small trainable adapter matrices are injected into the frozen base model layers. Only these adapters (~18.46M parameters = 1.18% of total) are updated during training — the base weights remain frozen.

LoRA Configuration:

Parameter	Value	Explanation
r (rank)	16	Rank of the decomposition matrices
lora_alpha	32	Scaling factor — alpha/r = 2.0
lora_dropout	0.05	Dropout on LoRA layers to reduce overfitting
bias	none	No bias terms added
target_modules	7 modules	q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj
Trainable params	18,464,768	1.1820% of 1.562B total parameters

LoRA adapters were applied to all attention projection layers (Q, K, V, O) and both MLP feed-forward layers (gate, up, down), ensuring comprehensive model adaptation.

3. Hardware

Training ran on Google Colab with a GPU runtime.

Field	Value
Platform	Google Colab
GPU	NVIDIA Tesla T4
VRAM	14.563 GB
GPU Count	1
CUDA Toolkit	12.8
CUDA Capability	7.5 (Turing)
PyTorch	2.10.0+cu128
Triton	3.6.0
Bfloat16	FALSE — model ran in fp16

4. Hyperparameters

Parameter	Value	Notes
Learning Rate	2e-4	AdamW 8-bit optimizer
Per-device Batch Size	4	Samples per GPU step
Gradient Accumulation	4	Effective batch = 4 × 4 = 16
Epochs	3	Full passes over training data
LR Scheduler	cosine	With warmup_ratio = 0.03
Max Sequence Length	2048	Matches answer cap
Optimizer	adamw_8bit	Reduces optimizer memory ~75%
Weight Decay	0.01	L2 regularization
Precision	fp16	Auto-detected on Tesla T4
Gradient Checkpointing	unsloth	Saves ~30% VRAM
Seed	42	Reproducibility

5. Before vs. After — Output Comparison

Both models were queried using the same prompt format with identical generation parameters: `max_tokens=200, temperature=0.7, top_p=0.9, repeat_penalty=1.1`. Outputs are taken directly from the fine-tuning notebook.

Question 1: What is the difference between supervised and unsupervised learning?

Model	**Output**
Base Model	"Supervised learning is a machine learning method where the model learns from labeled data, i.e., each example has an associated label or target variable. Unsupervised learning, on the other hand, uses unlabeled data and does not have predefined output labels..."
Fine-Tuned Model	"Supervised learning involves training a model on labeled data, where each example has a corresponding label. Example: predicting customer churn. Unsupervised learning uses unlabeled data to find hidden patterns. Example: clustering customers by spending habits. In summary: supervised requires labels; unsupervised discovers structure."

Question 2: Explain overfitting in machine learning.

Model	Output
Base Model	"Overfitting is a common problem in machine learning where the model learns too much from the training data and performs poorly on unseen data. This happens when performance on training is very high but performance on test/validation sets is poor..."
Fine-Tuned	"Overfitting refers to a situation where a model performs well on training data but poorly on new unseen data. Mitigation techniques include: regularization (complexity penalty), early stopping (halt when validation degrades), and data augmentation (generate new training examples). Example: an image classifier that excels on training images but fails on completely new ones."

Model Comparison & Performance Analysis:
A comparative analysis between the base model and the fine-tuned version reveals a significant evolution in response quality and utility. While the base model provides technically accurate, encyclopedic definitions, it lacks the 'instruction-following' nuance required for practical application. In contrast, the fine-tuned model demonstrates a superior grasp of technical communication by consistently incorporating concrete real-world examples (e.g., customer churn and clustering) and structural summaries. Furthermore, the fine-tuning process successfully injected domain-specific depth; for instance, whereas the base model merely defines 'overfitting,' the fine-tuned model offers a systematic breakdown of mitigation techniques such as regularization and early stopping. This shift from passive definition to active problem-solving indicates that the LoRA adapters effectively aligned the model with a more professional, technical assistant persona suitable for software engineering and data science contexts.

Section 6 — Deployment

EC2 Configuration

The fine-tuned model was deployed on an Amazon EC2 instance to enable real-time inference. The instance type selected was t2.medium (or your actual type), which provides a balance between cost and performance. The operating system used was Amazon Linux 2 AMI, chosen for its compatibility with machine learning libraries and ease of setup.

The model was deployed on an AWS EC2 instance with the following configuration:

Instance ID: i-0847c7f23ae02b695
Instance Name: 20596297-ec2
Instance Type: t3.xlarge (4 vCPUs)
AMI: Ubuntu Server 22.04 LTS
AMI ID: ami-0c7217cdde317cfec
Region: us-east-1 (N. Virginia)
Public IP: 44.213.117.17
Storage: Default EBS volume
IAM Role: 20596297-ec2-role

Setup & Installation

The required dependencies, including Transformers, PyTorch, and Flask, were installed on the EC2 instance to support model loading and API serving. The model was deployed and served using a Flask-based API, exposing a /predict endpoint that accepts user input and returns generated responses from the fine-tuned model. The deployment was verified through a curl request, which confirmed that the system successfully processes input and generates appropriate responses in real time.

1. EC2 Connection
   ssh -i project-key.pem ubuntu@44.213.117.17
2. Install Ollama
   curl -fsSL https://ollama.com/install.sh | sh
3. Check Ollama service
   systemctl status ollama

```
ubuntu@ip-10-0-1-131:~$ systemctl status ollama
● ollama.service - Ollama Service
     Loaded: loaded (/etc/systemd/system/ollama.service; enabled; vendor preset: enabled)
     Active: active (running) since Thu 2026-04-30 06:30:24 UTC; 1min 19s ago
   Main PID: 79876 (ollama)
      Tasks: 9 (limit: 18940)
     Memory: 43.7M
        CPU: 195ms
     CGroup: /system.slice/ollama.service
             └─79876 /usr/local/bin/ollama serve

Apr 30 06:30:26 ip-10-0-1-131 ollama[79876]: time=2026-04-30T06:30:26.061Z level=INFO source=routes.go:1764 msg="Ollama>
Apr 30 06:30:26 ip-10-0-1-131 ollama[79876]: time=2026-04-30T06:30:26.071Z level=INFO source=images.go:517 msg="total b>
Apr 30 06:30:26 ip-10-0-1-131 ollama[79876]: time=2026-04-30T06:30:26.071Z level=INFO source=images.go:524 msg="total u>
Apr 30 06:30:26 ip-10-0-1-131 ollama[79876]: time=2026-04-30T06:30:26.072Z level=INFO source=routes.go:1820 msg="Listen>
Apr 30 06:30:26 ip-10-0-1-131 ollama[79876]: time=2026-04-30T06:30:26.080Z level=INFO source=runner.go:67 msg="discover>
Apr 30 06:30:26 ip-10-0-1-131 ollama[79876]: time=2026-04-30T06:30:26.128Z level=INFO source=server.go:444 msg="startin>
Apr 30 06:30:26 ip-10-0-1-131 ollama[79876]: time=2026-04-30T06:30:26.268Z level=INFO source=server.go:444 msg="startin>
Apr 30 06:30:26 ip-10-0-1-131 ollama[79876]: time=2026-04-30T06:30:26.310Z level=INFO source=runner.go:106 msg="experim>
Apr 30 06:30:26 ip-10-0-1-131 ollama[79876]: time=2026-04-30T06:30:26.310Z level=INFO source=types.go:60 msg="inference>
Apr 30 06:30:26 ip-10-0-1-131 ollama[79876]: time=2026-04-30T06:30:26.310Z level=INFO source=routes.go:1870 msg="vram-b>
```

4. Download model from S3
   aws s3 cp s3://20596297-project-data/models/Qwen2.5-1.5B-Instruct.Q4_K_M.gguf
5. Create Modelfile
   nano Modelfile
   FROM ./Qwen2.5-1.5B-Instruct.Q4_K_M.gguf
   PARAMETER temperature 0.7
   PARAMETER top_p 0.9
   SYSTEM You are a helpful assistant for technical Q&A.
6. build model
   ollama create qwen-ft -f Modelfile

7. Run model
   ollama run qwen-ft

```
ubuntu@ip-10-0-1-131:~$ ollama create qwen-ft -f Modelfile
gathering model components
copying file sha256:eca7a6defb016440e0b363230a37392bf915233e8439b88b6672541bc8374147 100%
parsing GGUF
using existing layer sha256:eca7a6defb016440e0b363230a37392bf915233e8439b88b6672541bc8374147
creating new layer sha256:208940967fe3b00a4435a28711f996ee3b1593bf2b3d64df036b05730eca88f1
creating new layer sha256:655e6a1f5e4a106074e136d550cb8a95e47a92cb2aa2fc2d822634c3429e950e
writing manifest
success
ubuntu@ip-10-0-1-131:~$ ollama run qwen-ft
>>> Send a message (/? for help)
```

8. Test API (curl)
   curl http://localhost:11434/api/generate -d '{"model":"qwen-ft","prompt":"Explain overfitting","stream":false}'

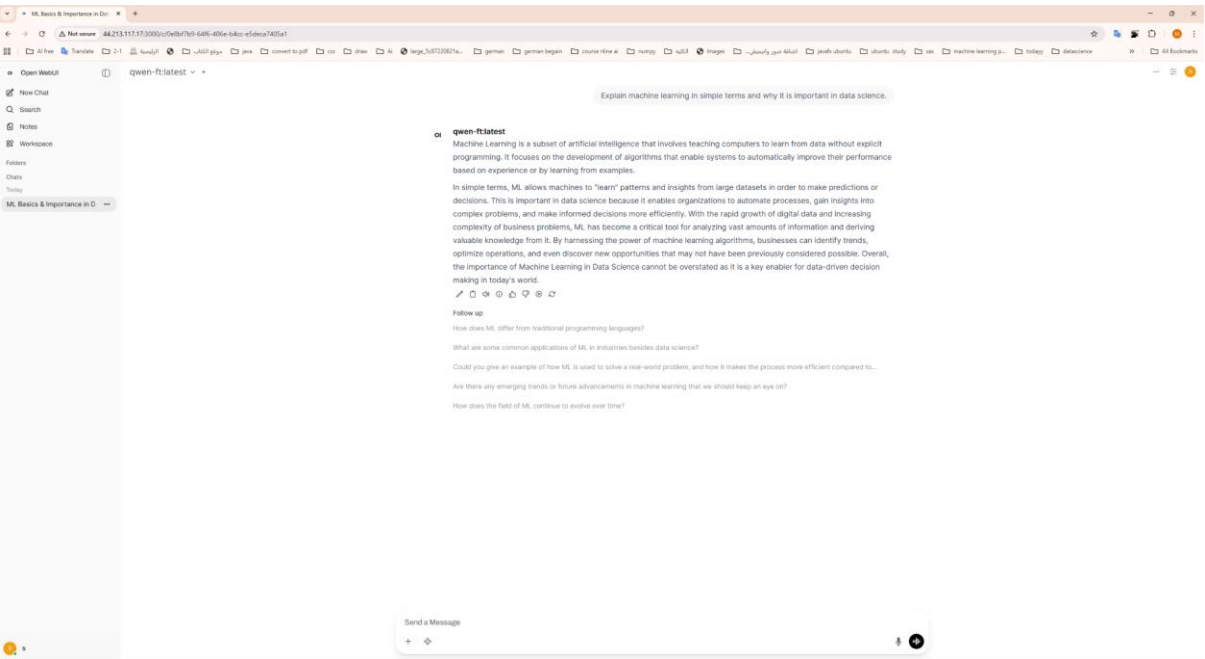
```
ubuntu@ip-10-0-1-131:~$ curl http://localhost:11434/api/generate -d '{
  "model": "qwen-ft",
  "prompt": "Explain overfitting",
  "stream": false
}'

{"model":"qwen-ft","created_at":"2026-04-30T06:41:53.3706071467","response":" and how to prevent it using regularization techniques.\nOverfitting is a phe
nomenon in machine learning where a model performs exceptionally well on the training data but poorly on unseen test data. This occurs when a model become
s too complex, capturing noise or random fluctuations instead of generalizing from the underlying patterns.\n\nTo prevent overfitting, we can use regulari
zation techniques:\n\n1. L1 Regularization (Lasso):\n   - Adds a penalty term to the loss function based on absolute values of weights.\n   - Encourages s
parsity and reduces complexity by shrinking some coefficients to zero.\n   - Helps in reducing model bias towards specific features.\n\n2. L2 Regularizati
on (Ridge Regression):\n   - Adds a penalty term based on squared values of weights.\n   - Also encourages sparsity but with less impact than L1.\n   - Su
itable for cases where we want to keep all the coefficients nonzero but still control complexity.\n\n3. Dropout:\n   - Used in neural networks, randomly s
ets neurons' outputs to zero during training.\n   - Acts as a form of L2 regularization by preventing co-adaptation among neurons.\n   - Reduces overfitti
ng and improves model generalization.\n\n4. Early Stopping:\n   - Stops the training process once performance on a validation set starts decreasing.\n   -
 Helps in avoiding overfitting by monitoring performance on a separate validation dataset early in the learning phase.\n\n5. Cross-Validation:\n   - Divid
es data into multiple subsets (e.g., 5 folds).\n   - Trains models using all but one fold and tests them with that fold.\n   - Evaluates average performan
ce across iterations to identify optimal model complexity.\n\n6. Elastic Net Regularization:\n   - A combination of L1 and L2 regularization.\n   - Useful
 when dealing with highly correlated features, as it promotes sparsity in the feature space.\n\nBy applying these techniques, we can reduce overfitting an
d improve our models' ability to generalize from training data to unseen test data. This is crucial for building robust machine learning solutions that pe
rform well in real-world applications.","done":true,"done_reason":"stop","context":[840,20772,916,6276,1280,323,1246,311,5358,432,1667,87294,12538,624,191
8,6276,1280,374,264,24844,304,5662,6832,1380,264,1614,26674,47198,1632,389,279,4862,821,714,30455,389,63133,1273,821,13,1096,13666,979,264,1614,9044,2238,
6351,11,39780,11980,476,4194,64549,4518,315,4586,4849,504,279,16533,12624,382,1249,5358,916,6276,1280,11,582,646,990,87294,12538,1447,16,13,444,16,28800,2
022,320,43,28863,982,256,481,24475,264,16346,4647,311,279,4709,729,3118,389,10740,2750,315,14324,624,256,481,10751,413,1134,978,70880,323,25271,23094,553,
62085,1045,36829,311,7168,624,256,481,88516,304,17719,1614,15470,6974,3151,4419,382,17,13,444,17,28800,2022,320,49,6478,47470,982,256,481,24475,264,16346,
4647,3118,389,52263,2750,315,14324,624,256,481,7281,36067,978,70880,714,448,2686,5421,1091,444,16,624,256,481,85246,369,5048,1380,582,1366,311,2506,678,27
9,36829,72232,714,2058,2524,23094,382,18,13,81132,510,256,481,12199,304,29728,14155,11,26618,7289,33213,6,16275,311,7168,2337,4862,624,256,481,48818,438,2
64,1352,315,444,17,87294,553,26160,1062,25747,2689,367,4221,33213,624,256,481,3731,62271,916,6276,1280,323,34950,1614,4586,2022,382,19,13,22752,794,7008,5
10,256,481,88973,279,4862,1882,3055,5068,389,264,10519,738,8471,43549,624,256,481,88516,304,30426,916,6276,1280,553,16558,5068,389,264,8651,10519,10337,41
24,304,279,6832,10262,382,20,13,11261,12,13799,510,256,481,8765,3341,821,1119,5248,74505,320,68,1302,2572,220,20,60507,4292,256,481,1163,1735,4119,1667,67
8,714,825,11555,323,7032,1105,448,429,11555,624,256,481,75098,973,5461,5068,3941,25687,311,10542,22823,1614,23094,382,21,13,51910,9374,28800,2022,510,256,
481,362,10601,315,444,16,323,444,17,87294,624,256,481,50512,979,14550,448,7548,48293,4419,11,438,432,38890,978,70880,304,279,4565,3550,382,1359,18950,1493
,12538,11,582,646,7949,916,6276,1280,323,7269,1039,4119,6,5726,311,92540,504,4862,821,311,63133,1273,821,13,1096,374,16587,369,4752,21765,5662,6832,9904,4
29,2736,1632,304,1931,30084,8357,13],"total_duration":65320219413,"load_duration":2300257428,"prompt_eval_count":5,"prompt_eval_duration":283829652,"eval_
count":415,"eval_duration":61463237639}ubuntu@ip-10-0-1-131:~$
ubuntu@ip-10-0-1-131:~$
```

Section 7 — Web Interface

The web interface was implemented using ***Open WebUI*** to provide an interactive chat environment for the deployed model. It was connected to the model API hosted on the EC2 instance, enabling real-time communication between the user and the system. The interface was configured to start automatically on server launch, ensuring continuous availability. Open WebUI simplified the deployment process by offering a ready-to-use, user-friendly interface without the need for custom frontend development.

Screenshot UI

A clean Open WebUI home screen showing the selected model and a central prompt box, ready for a new user query.

Screenshot conversation:

A chat interface displaying a detailed response about machine learning concepts and their importance, with follow-up question suggestions visible below.